

AI-Driven Network Traffic Analysis and Intrusion Detection System

Technical Report

1. Problem Overview and Objectives

Modern enterprise networks face evolving threats, including volumetric Denial of Service (DoS) attacks, stealthy reconnaissance, and lateral movement. Traditional signature-based detection mechanisms (e.g., Snort) are highly effective against known attack signatures but routinely fail against zero-day exploits or obfuscated traffic.

The objective of this project is to design and deploy a working Machine Learning (ML) prototype capable of ingesting network flow data, extracting meaningful behavioral features, and classifying traffic as benign or malicious. Crucially, the system must provide explainable outputs to support SOC analysts during incident triage.

2. System Architecture & Feature Extraction

The system architecture consists of a Python ingestion engine, a Scikit-Learn inference layer, and a Command-Line Interface (CLI) for analyst interaction.

Feature extraction is the most critical component of behavioral anomaly detection. Performing Deep Packet Inspection (DPI) on payloads is computationally expensive and fails on encrypted traffic. Therefore, this system extracts 14 statistical flow features. The most heavily weighted features utilized by the model include:

- **Flow Bytes/s & Fwd Packets/s:** Highly indicative of volumetric flooding and DDoS.
- **Flow Duration:** Crucial for identifying "low and slow" reconnaissance or beaconing.
- **Packet Length Mean:** Deviations here often indicate payload delivery, exploitation, or data exfiltration.

3. Detection Methodology

A supervised Machine Learning approach utilizing a **Random Forest Classifier** was implemented.

- **Model Justification:** Random Forest was selected because it handles tabular network flow data exceptionally well without requiring complex feature scaling. It creates multiple decision trees and aggregates their output, making it highly resistant to overfitting on specific network noise.
- **Explainability:** Random Forest algorithms inherently calculate *Feature Importance*. This fulfills the critical requirement for explainability; when the system flags an event, it does not act as a "black box." The CLI evaluates the most heavily weighted features (e.g., an abnormal spike in packet frequency) and outputs a human-readable reason for the alert.

4. Testing, Evaluation, and Results

The model was evaluated against five practical demonstration scenarios, yielding the following results:

- **Scenario 1 (Benign Baseline):** Correctly classified Normal Traffic with 100.00% confidence. Maintaining a near-zero false-positive rate on baseline traffic is critical to prevent alert fatigue.
- **Scenario 2 (PortScan):** Detected anomalous reconnaissance with 79.50% confidence. The system accurately cited "long flow duration with extremely low byte rate" as the reason,

matching the signature of stealthy scanning.

- **Scenario 3 (DDoS):** Correctly classified with 100.00% confidence. The system flagged an abnormal mean packet length (1058 bytes), successfully detecting the volumetric deviation.
- **Scenario 4 (Brute Force / FTP-Patator):** This scenario yielded a misclassification. The model flagged the traffic as a Web Attack (XSS) with a low confidence of 54.75%. This highlights a reality of ML models: credential stuffing over standard ports shares heavy statistical flow similarities with noisy web application traffic.
- **Scenario 5 (Infiltration / Ambiguous Case):** Detected anomalous behavior with 65.00% confidence. Infiltration and lateral movement are inherently designed to mimic benign internal traffic. The low confidence score exposes the difficulty of detecting lateral movement strictly through flow statistics without host-based logs.

5. Network and Security Considerations (Limitations)

Deploying this AI-driven approach in a production enterprise network introduces several limitations:

- **Performance Impact:** Real-time, inline ML detection introduces latency and throughput bottlenecks. To scale to high-throughput networks (10Gbps+), this system should be deployed out-of-band, analyzing SPAN port traffic or NetFlow logs retroactively.
- **Adversarial Evasion:** Threat actors aware of behavioral IDS can utilize evasion techniques. By artificially adding delays between packets, adversaries can force their malicious traffic to fall within the acceptable benign threshold parameters, effectively evading threshold-based detection.
- **False-Positive Impact:** A strict anomaly model can cause operational disruption. If a legitimate user initiates a massive file transfer, the Flow Bytes/s will spike, potentially triggering a false positive and wasting analyst time.

6. Future Improvements

To improve precision, a hybrid approach should be adopted. The ML model should be paired with a signature-based engine (like Snort) to catch known CVEs instantly with zero compute overhead, leaving the ML model strictly to hunt for unknown, zero-day volumetric anomalies.

7. Technology Selection & Justification

To ensure a highly reproducible and performant prototype, the following technology stack was selected:

- **Programming Language:** Python 3. Python was chosen for its unparalleled ecosystem in data manipulation (pandas, numpy) and machine learning, allowing for rapid prototyping of behavioral models.
- **Detection Framework:** Scikit-Learn. Scikit-learn provides robust, enterprise-grade implementations of ensemble learning algorithms like Random Forest. It is lightweight and does not require the massive computational overhead of deep learning frameworks like TensorFlow or PyTorch.
- **Dataset (Traffic Data):** CIC-IDS-2017 (Canadian Institute for Cybersecurity). Rather than generating synthetic traffic in a Mininet topology, a public dataset was chosen. Generating isolated traffic via Mininet often lacks the complex, noisy background traffic of a real enterprise. CIC-IDS-2017 provides millions of labeled flows containing realistic background traffic interwoven with modern attack profiles.

- **Traffic Capture/Analysis:** Rather than using Wireshark or tcpdump for live PCAP analysis, this prototype ingests pre-processed flow statistics (CSV format). This mimics the behavior of enterprise flow-exporters (like Zeek or Cisco NetFlow) sending aggregated metadata to a SIEM.

8. Setup and Execution Instructions

To reproduce these results, follow the instructions below:

Prerequisites:

- Python 3.8+
- The CIC-IDS-2017 dataset CSV files (specifically Friday-DDos, Friday-PortScan, Tuesday-WorkingHours, and Thursday-Infiltration).

Installation:

1. Clone the repository to your local machine.
2. Install dependencies: `pip install pandas numpy scikit-learn joblib`
3. Place the dataset CSVs into the `datasets/MachineLearningCVE/` directory.

Execution: Run the CLI demonstration script from the root directory: `python src/run_scenarios.py`

The system will ingest the traffic, load the pre-trained `attack_classifier.pkl`, and output the threat classifications, confidence scores, and feature-driven explanations to the terminal.